
SYMBOLIC REGRESSION CAN PERFORM ON PAR WITH TREE-BASED ENSEMBLES IN REGRESSION PROBLEMS

Alberto Tonda, Evelyne Lutton

Applied Mathematics and Computer Science Lab - Paris-Saclay (UMR 518 MIA-PS), Palaiseau, France

Complex Systems Institute of Paris Île-de-France (UAR 3611 ISC-PIF), Paris, France

Université Paris-Saclay, INRAE, CNRS

{alberto.tonda, evelyne.lutton}@inrae.fr

ABSTRACT

In machine learning, the state of the art for regression problems is currently represented by black-box ensembles of decision trees, such as Random Forest and eXtreme Gradient Boosting. Symbolic Regression, an Evolutionary Computation technique, can tackle regression problems and deliver human-readable models in the form of equations. However, the consensus in the field is that Symbolic Regression is generally less effective than tree-based approaches. Providing a fair comparison of these techniques is not straightforward, as they employ extremely different paradigms. In this work, we propose an empirical evaluation of tree-based ensembles and Symbolic Regression, employing state-of-the-art software libraries compared on a curated regression benchmark suite, using rigorous statistical tests to assess differences in performance. The results reveal that Symbolic Regression with commonly used default hyperparameters is indeed generally less performant than tree-based ensembles. Nevertheless, we show that when the final model selection method is changed and more computational resources are allocated to all techniques, Symbolic Regression becomes more competitive, often performing on par with the state of the art. Finally, we verify the interpretability of the models discovered by Symbolic Regression for selected problems: An expert analysis of the equations discovered during the 10-fold cross-validation uncovers common structural elements, eventually delivering a global form.

Keywords Benchmarking · Extreme Gradient Boosting · Random Forest · Regression · Symbolic Regression

1 Introduction

Given a dataset $\mathcal{D} = \{(X_i, y_i)\}_{i=0}^s$, where $X_i \in \mathbb{R}^n$ is a vector of n input features and $y_i \in \mathbb{R}$ is the ground truth value for each of the s samples, regression algorithms aim to find a function $\hat{y} = f(X)$ such that $y_i \approx \hat{y}_i, \forall i$. A considerable variety of approaches have been developed to tackle regression problems, from linear regression [20] to Support Vector Machines [7], from Decision Trees (DTs) [4] to ensembles of DTs such as Random Forest (RF) [3] and eXtreme Gradient Boosting (XGBoost) [6].

While most of the high-performing approaches originate from the field of Machine Learning (ML), at least one regression algorithm emerged from the domain of Evolutionary Computation (EC) and Genetic Programming (GP): Symbolic Regression (SR) [15], able to stochastically explore the space of free-form equations to discover interpretable models. Other techniques, such as Deep Learning, have also been employed to perform equation discovery [25], but in this work we will consider just the GP-based version, as it currently represents the dominant paradigm. Modern GP approaches to SR [5, 8, 23] have been successful in finding interpretable models for a wide range of applications, from industrial fouling [14], to bacterial growth [24], to associations between biomarkers and diseases [22].

Despite a remarkable history of favorable results, the general consensus among industry and practitioners for tabular regression problems is that SR is not among the state of the art in the domain, currently considered to be represented by tree-based ensembles such as RF and XGBoost [10]. For a long time, the main obstacle to a wider adoption of SR was the absence of software libraries usable out of the box, such as `scikit-learn` [21] for ML algorithms; but in recent

years, several highly curated software solutions emerged, for example Operon [5] and PySR [8]. Another obstacle to the popularity of SR is the higher computational cost, order of magnitudes higher than RF or XGBoost; SR libraries often set their default hyperparameters so that SR terminates in a few seconds, a time comparable to other approaches, but this can go against the final quality of the results. Finally, modern SR software employs a complexity/fitting Pareto approach, but often the user just wants one single solution; and selecting a candidate model on the front is not easy, as heuristics might deliver under-performing results.

The motivation of this work is to design and carry out a proper, thorough experimental comparison of SR against modern regression algorithms, to verify whether SR can perform on par with the state of the art. To achieve our objective, we explore the following research questions:

- RQ1** Is it possible to design an experimental comparison which is as fair as possible for both SR and tree-based models, allowing them to efficiently exploit the same amount of computational resources?
- RQ2** Given enough computational resources, can SR be competitive with tree-based models for regression problems?
- RQ3** For SR, the final model selection on the complexity/fitting front is pivotal for the quality of the results. Is there a significant difference between using heuristics (as in modern SR libraries) and exploiting a validation set?

Tackling **RQ1** is not straightforward, as differences in algorithms (iterative versus parallel), implementation, and programming language can have significant impacts. As a proposed solution, we design an experimental evaluation around the OpenML-CTR23 benchmark suite [9], using the wall-clock time employed by SR to perform hyperparameter tuning for the tree-based models, with the hyperparameter optimization library Optuna [1].

The answer to **RQ2** is positive: During the experimental evaluation, the state-of-the-art SR library PySR [8] is often able to perform on par with RF and XGBoost, even when hyperparameter tuning for the other algorithms is involved. However, obtaining this result comes with a considerable increase in computational time, allowing PySR to generate equations of large size (thus losing interpretability), and running it for a large amount of iterations.

Finally, comparison of SR models selected using PySR internal heuristic and a validation set carved out from the training data (thus reducing the amount of samples available for pure training) shows the second approach to be vastly superior, answering **RQ3** and providing a useful recommendation for practitioners interested in using SR for real-world problems.

As one of the main stated advantages of SR over other ML approaches is the interpretability of its results, in the last part of the paper we present the equations discovered by SR for selected datasets in the benchmark suite, showing how it is possible to eventually find a common form through an expert analysis, and obtain compact, interpretable, highly predictive models.

2 Background

2.1 Symbolic regression

Symbolic Regression (SR) is a technique designed for regression problems, from the field of Evolutionary Computation (EC), and Genetic Programming (GP) [15] in particular. SR discovers analytical mathematical expressions that describe underlying data patterns. Unlike traditional linear or quadratic regression, which fit parameters to a predefined functional form, SR searches the combinatorial space of mathematical operators, constants, and variables, to identify the model structure itself. Modern approaches to SR introduced improvements in the operators and the selection, for example preserving a set of equations each representing a different compromise on the complexity/error front [23]. After the initial release of commercial SR software such as Eureqa® Formulize in the 2010s, in recent years the SR community released highly optimized open-source software libraries, such as Operon [5] and PySR [8].

2.2 Ensembles of decision trees

Tree-based models are currently considered the state of the art for tabular regression [10]. Such approaches consistently outperform Deep Learning and other statistical models on standard tabular benchmarks. While recent Transformer-based approaches (such as TabPFN) have reported competitive or superior performance [12], we decided to not include them in this study, since (i) the benchmark suite employed here is likely to have been part of its training set, and (ii) a pre-trained model would not benefit from the hyperparameter optimization strategy described in the following.

A Decision Tree (DT) addresses regression by recursively partitioning the feature space into axis-aligned regions through a sequence of feature-threshold tests. Each terminal leaf is associated with a prediction value. Splits are selected

Task ID	Dataset name	Samples	Features	Missing values	Categorical features
361234	abalone	4,177	8	0	1
361235	airfoil_self_noise	1,503	5	0	0
361236	auction_verification	2,043	7	0	2
361237	concrete_compressive_strength	1,030	8	0	0
361241	physiochemical_protein	45,730	9	0	0
361242	superconductivity	21,263	81	0	0
361243	geographical_origin_of_music	1,059	116	0	0
361244	solar_flare	1,066	10	0	8
361247	naval_propulsion_plant	11,934	14	0	0
361249	white_wine	4,898	11	0	0
361250	red_wine	1,599	11	0	0
361251	grid_stability	10,000	12	0	0
361252	video_transcoding	68,784	18	0	2
361253	wave_energy	72,000	48	0	0
361254	sarcos	48,933	21	0	0
361255	california_housing	20,640	8	0	0
361256	cpu_activity	8,192	21	0	0
361257	diamonds	53,940	9	0	3
361258	kin8nm	8,192	8	0	0
361259	pumadyn32nh	8,192	32	0	0
361260	miami_housing	13,932	15	0	0
361261	cps88wages	28,155	6	0	4
361264	scomob	1,156	6	0	4
361266	kinges_county	21,613	21	0	4
361267	brazilian_houses	10,692	9	0	4
361268	fps_benchmark	24,624	43 (39)	69,696	13
361269	health_insurance	22,272	11	0	7
361272	4ifa	19,178	28	0	1
361616	Moneyball	1,232	14 (10)	3,600	4
361617	energy_efficiency	768	8	0	0
361618	forest_fires	517	12	0	2
361619	student_performance_por	649	30	0	17
361621	QSAR_fish_toxicity	908	6	0	0
361622	cars	804	17	0	0
361623	espace_ga	3,107	6	0	0

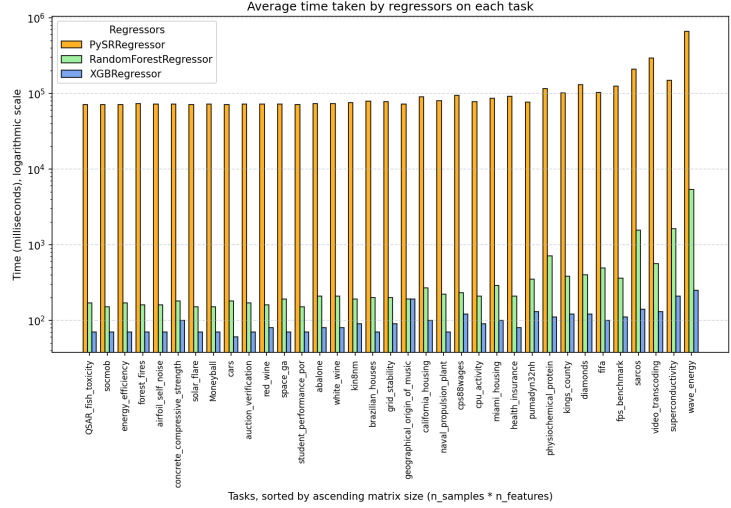


Table 1: Regression tasks and related datasets included in the OpenML-CTR23 benchmark suite. Two datasets (fps_benchmark and Moneyball) have features with missing values, which will be removed during preprocessing.

Figure 1: Wall-clock time (milliseconds) taken by the considered algorithms on the different datasets. Datasets are sorted by size of the feature matrix (number of samples times number of features).

to minimize a loss or impurity criterion, but unconstrained trees can easily overfit unless regularized through depth limits, minimum leaf size, or pruning. To improve performance, modern approaches rely on ensembles of DTs.

Random Forest (RF) [3] constructs multiple trees using bootstrap sampling of the training data and random subsets of features at each split. Predictions are then aggregated across trees, typically by averaging for regression or majority voting for classification, which reduces variance and improves generalization. eXtreme Gradient Boosting (XGBoost) [6] builds trees sequentially, with each new tree trained to correct the residual errors of the current ensemble by optimizing a differentiable loss function. It combines regularization with efficient optimization strategies to improve predictive accuracy while controlling overfitting.

RF and XGBoost often achieve remarkable predictive performance by combining a large number of weak learners. However, this predictive strength comes at the cost of interpretability: unlike a single symbolic expression, such ensembles cannot be directly analyzed, algebraically manipulated, or checked for dimensional consistency. By contrast, SR provides interpretability through explicit analytical expressions, making it attractive when model transparency is required.

3 Proposed approach

3.1 The OpenML-CTR23 benchmark suite

While the SR community has also developed benchmarks suites for regression problems [16, 19], in this study we decided to use the OpenML-CTR23 benchmark suite [9], which is commonly used in the field of Machine Learning (ML). OpenML-CTR23 includes 35 curated datasets, handpicked from the larger OpenML repository¹ [2], with a number of samples ranging from 517 to 72,000, and a number of features between 5 and 116. Benchmarks include mostly real-valued features, with a number of categorical features ranging from 0 to 17. Each dataset in the benchmark is connected to a reference, and folds for a k -fold cross-validation (typically with $k = 10$) are pre-computed to improve fairness in comparison. The datasets in the suite are detailed in Table 1.

3.2 Evaluating performance in regression

Regression performance can be assessed by evaluating metrics such as mean squared error and R^2 on the test sets of each fold. Assuming large differences in behavior between regressors and small standard deviation, comparing the

¹OpenML, <https://www.openml.org/>

mean R^2 on test could suffice; but in several cases, performances in state-of-the-art algorithms will be close. It is possible to establish whether a difference is statistically significant by employing hypothesis tests. In this setting, an algorithm’s performance on each fold is treated as a sample drawn from an underlying random distribution, and the test evaluates whether two sets of samples are likely to originate from the same distribution. The decision is then based on the p -value, denoted by p , that represents the probability of obtaining an observed difference at least as large as the measured one purely by chance, assuming that the null hypothesis is true, i.e., assuming that no real difference exists between the compared algorithms. A user-defined significance threshold, typically $\alpha = 0.05$, is then used: if $p < \alpha$, the null hypothesis is rejected and the difference is considered statistically significant.

Choosing the correct statistical test to apply is not straightforward, as a classic paired t -test for example assumes that the samples are independent; however, this is not true when performing a k -fold cross-validation, as most of the training set will overlap between different folds. To tackle this issue, we select a test specifically designed for such cases, the Nadeau-Bengio’s Corrected Resampled t -test [18], which introduces a correction factor to the variance calculation to account for the overlapping training data.

While Nadeau-Bengio works perfectly for evaluating the difference in performance between two regressors, a comparison between three or more regressors introduces additional subtleties. Selecting an arbitrary threshold α to establish significance means accepting a corresponding α chance of a false positive (Type I error) for a single test, for example $\alpha = 0.05$ would correspond to a $P(E) = 0.05$ chance of error; but since comparing multiple regressors implies performing several pairwise tests, with m regressors the true probability of obtaining at least one error becomes $P(E) = 1 - (1 - \alpha)^m$, going up to $P(E) = 0.143$ for $m = 3$. To solve this issue, we employ a Holm-Bonferroni correction [13].

3.3 Obstacles to a fair comparison

Ensuring fairness during the comparison of the different regression algorithms is quite challenging, for two reasons: (i) the inner working of each algorithm; (ii) the specific implementation.

SR and XGBoost are both iterative algorithms, while RF generates all its trees in parallel, at the same time. Furthermore, SR requires a full evaluation of each candidate solution generated, while RF and XGBoost both employ a greedy strategy at the level of each single leaf, relying upon faster assessments. Given these premises, it is easy to see how SR would benefit the most from a larger amount of available computational resources.

Since we are interested in comparing the state of the art, we rely upon established libraries for each of the considered algorithms. While the most proper way of ensuring fairness would be to re-implement all algorithms in the same programming language, the effort required would be immense, and the risk of missing important details introduced in the most up-to-date libraries would be extremely high. Using existing software, however, comes with several disadvantages.

The RF implementation employed in the study is `RandomForestRegressor` from the `scikit-learn` [21] Python module; for XGBoost, we use the `xgboost` Python module, which, however, relies on a compiled C++ core called with Python bindings; and the library chosen for SR is `PySR` [8], again with a Python interface but a core written in Julia. The speed of the scripts of an interpreted language like Python is clearly orders of magnitude slower than the compiled code of C++ or Julia: This creates issues when the aim is to fairly compare the algorithms.

To tackle these issues, we propose the following solutions. Given the differences in inner working and implementation of the algorithms involved in the comparison, we resort to measuring the wall-clock time taken by each approach on the same hardware, for each specific task. The time employed by the slowest algorithm (predictably SR) will then be used as a reference. Since SR can likely profit the most from the availability of more computational resources, the same amount of time used by SR will be devoted to performing hyperparameter optimization for RF and XGBoost. Hyperparameter optimization or hyperparameter tuning is the process of finding the values of the hyperparameters which deliver the best performance, typically assessed in a cross-validation or with a validation set.

4 Experimental evaluation

Following the ideas developed in the previous section, we perform two separate sets of experiments on OpenML-CTR23. In the first, all algorithms employ default hyperparameters of the respective libraries. In the second one, `PySR` uses enhanced hyperparameters, leading to a much higher computational cost and consequent longer wall-clock time; all other algorithms exploit the same amount of time to perform hyperparameter optimization with a state-of-the-art library.

All experiments are executed on the same hardware, an Intel® Xeon® w9-3475X with 36 cores and 128 GB of RAM. The code developed for the experimental evaluation, in Python, is available on a public GitHub repository².

4.1 Preprocessing

OpenML-CTR23 already has pre-computed splits for k -fold cross-validation, which is always with $k = 10$; for a few datasets a 10x10-fold cross-validation is defined, giving the possibility of performing the 10-fold cross-validation 10 times; but in all those cases, we just used the first 10-fold division. Feature columns with missing values have been dropped, which is only relevant for the two datasets `fps_benchmark` (going from 43 to 39 features) and `Moneyball` (going from 14 to 10 features). Values of categorical features have been converted to consecutive integer values, in order of appearance. When a validation set was used, for both PySR model selection and tree-based model hyperparameter tuning, the same 20% of the training set was randomly assigned to validation. No normalization was performed on the features, as both SR and tree-based approaches typically do not require values in a specific range to be effective.

4.2 Default hyperparameters

In a first experimental run, PySR with default hyperparameters (as of version 1.5.9) is tested on a 10-fold cross-validation on all datasets in OpenML-CTR23. As most SR libraries, PySR has a considerable number of hyperparameters; here we report only the values that will be changed in the following, which are also the most impactful for the quality of the final results. `binary_operators=[+, -, *, /]` describes all operators with two arguments; `unary_operators=[]`, on the other hand, includes operators such as `cos` and `log`, but by default the list is empty. `maxsize=30` is the maximum complexity of an equation, set as the sum of nodes in the tree. `iterations=100` is the maximum number of generations given to the evolutionary algorithm. PySR employs a multi-population scheme, by default with `npopulations=31` populations of `populationsize=27` individuals each, for a total population size of 837 candidate solutions. The default heuristic that PySR uses to select the final model between non-dominated solutions on the error/complexity front is by first computing a score corresponding to the negated derivative of the log-error with respect to complexity; and then by selecting the candidate model with the highest score among expressions with an error better than at least 1.5 times the most accurate model [8].

`RandomForestRegressor` and `XGBRegressor` also employ default hyperparameters from their respective libraries, with the most impactful for both being the number of trees in the ensemble `n_estimators=100`. By default, `RandomForestRegressor` expands trees until all leaves are pure, or each leaf contains `min_samples_split=2` samples or less. All features are considered for the splits, and the criterion used to optimize the trees is the mean squared error with respect to the training data. `XGBRegressor` has a higher number of hyperparameters, with the most relevant besides number of estimators being maximum tree depth `max_depth=6`, learning rate `eta=0.3`, and L2 regularization weight `lambda=1`.

4.2.1 Timing

Figure 1 reports the average time spent by each algorithm on a single fold of the cross-validation process, per dataset. Datasets are sorted by size of their training matrix (number of samples multiplied by the number of features). XGBoost shows a consistent performance, with time per fold ranging from 0.06 s on the smallest datasets to 0.25 s on the largest. Random Forest, which is implemented in Python and not in a compiled language, but that on the other hand is not an iterative algorithm, takes between 0.15 s and 5.35 s per fold. As expected, the SR algorithm is by far the slowest, with times ranging from 70.83 s to 654.05 s. As each candidate solution for SR needs to be tested on the available training samples, PySR’s computational effort increases as the number of samples grows. It is important to remark that SR can in principle employ batching, evaluating each individual only on a small subset of the training data, but for the experiments in this work we decided to not use this option, as we are interested in assessing how the algorithm scales as the number of samples in the training set varies.

4.2.2 Performance

As previously described in subsection 3.2, we employ the proper statistical tests to assess significant differences in performance. First, in order to explore **RQ3**, we are interested in comparing the two different methodologies for performing model selection in SR; we thus apply the pairwise Nadeau-Bengio test on the dataset, with results summarized in Table 2. Choosing a SR model using the validation set is always equal to or better than the heuristic, so in the following we will use only this modality.

²<https://github.com/albertotonda/symbolic-regression-vs-ensembles>

Task ID	Dataset name	PySR R2, mean and std		Significant difference	
		Heuristic	Validation	$p < 0.05$	$p < 0.01$
361234	abalone	0.5074 ± 0.0253	0.5652 ± 0.0229	Yes	Yes
361235	airfoil_self_noise	0.5217 ± 0.1020	0.6364 ± 0.0620	Yes	Yes
361236	auction_verification	0.8363 ± 0.0354	0.8773 ± 0.0274	Yes	Yes
361237	concrete_compressive_strength	0.7498 ± 0.0484	0.8092 ± 0.0271	Yes	Yes
361241	physiochemical_protein	0.1282 ± 0.0077	0.2890 ± 0.0091	Yes	Yes
361242	superconductivity	0.6065 ± 0.0330	0.7113 ± 0.0203	Yes	Yes
361243	geographical_origin_of_music	0.0878 ± 0.0521	0.0709 ± 0.0456	No	No
361244	solar_flare	0.0857 ± 0.0342	-1.5347 ± 4.8692	No	No
361247	naval_propulsion_plant	0.6511 ± 0.1264	0.6970 ± 0.1294	Yes	Yes
361249	white_wine	0.1858 ± 0.0387	0.2952 ± 0.0447	Yes	Yes
361250	red_wine	0.1650 ± 0.0926	0.3503 ± 0.0299	Yes	Yes
361251	grid_stability	0.6438 ± 0.0718	0.7000 ± 0.0456	Yes	No
361252	video_transcoding	0.7068 ± 0.0732	0.7734 ± 0.0376	Yes	Yes
361253	wave_energy	0.7957 ± 0.0474	0.7978 ± 0.0460	No	No
361254	sarcos	0.9108 ± 0.0049	0.9272 ± 0.0031	Yes	Yes
361255	california_housing	0.4937 ± 0.0990	0.6127 ± 0.0405	Yes	Yes
361256	cpu_activity	0.9637 ± 0.0098	0.9713 ± 0.0070	Yes	Yes
361257	diamonds	0.9400 ± 0.0521	0.7899 ± 0.4962	No	No
361258	kin8nm	0.3753 ± 0.1030	0.5241 ± 0.0327	Yes	Yes
361259	pumadyn32nh	0.4383 ± 0.2303	0.4451 ± 0.2350	Yes	No
361260	miami_housing	0.7056 ± 0.0474	0.7696 ± 0.0305	Yes	Yes
361261	cps88wages	0.0962 ± 0.0178	0.2724 ± 0.0403	Yes	Yes
361264	socmob	0.4331 ± 1.0089	0.7661 ± 0.0908	No	No
361266	kings_county	0.6649 ± 0.0513	0.7188 ± 0.0414	Yes	Yes
361267	brazilian_houses	0.4645 ± 0.3103	0.6718 ± 0.3171	No	No
361268	fps_benchmark	0.4811 ± 0.0938	0.6023 ± 0.0713	Yes	Yes
361269	health_insurance	0.2797 ± 0.0093	0.3659 ± 0.0137	Yes	Yes
361272	fifa	0.6794 ± 0.0308	0.7548 ± 0.0333	Yes	Yes
361616	Moneyball	0.9281 ± 0.0137	0.9453 ± 0.0044	Yes	No
361617	energy_efficiency	0.9481 ± 0.0355	0.9547 ± 0.0293	No	No
361618	forest_fires	-11.4194 ± 31.2189	-0.2514 ± 0.3384	No	No
361619	student_performance_por	0.0931 ± 0.0593	0.2208 ± 0.1170	Yes	No
361621	QSAR_fish_toxicity	0.4079 ± 0.0838	0.5559 ± 0.0843	Yes	Yes
361622	cars	0.8891 ± 0.0312	0.9136 ± 0.0308	Yes	Yes
361623	space_ga	0.4662 ± 0.0667	0.5484 ± 0.0347	No	No

Table 2: Comparison in performance of model selection approaches (heuristic choice versus using a validation set) for SR using PySR. If the difference is significant ($p < 0.05$), the model with the best R^2 score is highlighted in orange.

To start assessing the initial situation related to **RQ2**, we compare the performance of the regressors using default hyperparameters. The complete results are reported in Table 3, with two columns reporting whether one of the algorithms is statistically better using two arbitrary thresholds of significance ($\alpha = 0.05$ and $\alpha = 0.01$). One first interesting global consideration is that some of the tasks appear difficult for all regressors. R^2 values can range in $(-\infty, 1.0]$, with values close to 1.0 corresponding to the best predictions: The most performing algorithm for tasks such as 361618 (forest_fires) or 361244 (solar_flare), for example, has an R^2 value which is negative or barely above 0.0, indicating an extremely poor performance. This leads us to believe that for those datasets it might not be possible to efficiently predict the information for the target using the information contained in the features. In other cases, such as tasks 361619 (student_performance_por) or 361264 (socmob), all regressors show a considerable variance, indicating that some of the folds might be harder to tackle.

For $p < 0.05$, as expected, XGBoost is overall the significantly best algorithm for most tasks (10/35), followed by Random Forest (7/35), with SR last (3/35); leaving out 25/35 datasets for which there is no significantly best regressor. We also performed a second statistical analysis on each task, verifying whether it was possible to isolate one regressor as the significantly worst. For $p < 0.01$, the least performing regressor was SR (16/35), with RF under-performing on 2/35 datasets and XGBoost on 1/35; for 16/35 datasets, there is no clear worst performer. From these results, it is clear that with default hyperparameters SR is not only slower, but also less performing than the tree-based models. In the next set of experiments, we aim to verify whether it is possible to improve SR’s performance by giving the algorithm more time to explore a vaster search space of equations, while simultaneously allowing the tree-based models to improve their performance by tuning hyperparameters for the same amount of time.

4.3 Enhanced hyperparameters

In a second experimental evaluation, PySR is manually set with hyperparameter values leading to a larger search space and more computationally expensive runs: `binary_operators=[+, -, *, /]`, `unary_operators=[sin, cos, log, exp]`, `maxsize=300`, `populationsize=100` (for a total number of 3,100 individuals across all populations), `niterations=1000`. The average wall-clock time for each split in the same dataset (now much larger than in the previous experiments) is then used as a timeout for performing hyperparameter tuning on RF and XGBoost using Optuna [1]. We set the maximum allowed time for PySRRegressor to 24 minutes of wall-clock time per fold, to ensure that a full 10-fold cross-validation using all regressors will be completed in 12 hours, and thus a full experimental run

Task ID	Dataset name	Symbolic Regression	R2, mean and std		XGBoost	Significant best		Significant worst	
			Random Forest			$p < 0.05$	$p < 0.01$	$p < 0.05$	$p < 0.01$
361234	abalone	0.5652 ± 0.0229	0.5443 ± 0.0177	0.4919 ± 0.0282	SR	-		XGB	XGB
361235	airfoil_self_noise	0.6364 ± 0.0620	0.9292 ± 0.0151	0.9466 ± 0.0152	XGB	XGB		SR	SR
361236	auction_verification	0.8773 ± 0.0274	0.9928 ± 0.0015	0.9983 ± 0.0005	XGB	XGB		SR	SR
361237	concrete_compressive_strength	0.8092 ± 0.0271	0.8995 ± 0.0274	0.9232 ± 0.0240	XGB	XGB		SR	SR
361241	physiochemical_protein	0.2890 ± 0.0091	0.6624 ± 0.0057	0.6015 ± 0.0057	RF	RF		SR	SR
361242	superconductivity	0.7113 ± 0.0203	0.9217 ± 0.0074	0.9176 ± 0.0066	RF	RF		SR	SR
361243	geographical_origin_of_music	0.0709 ± 0.0456	0.2623 ± 0.0922	0.1841 ± 0.0935	RF	-		SR	-
361244	solar_flare	-1.5347 ± 4.8692	-0.0980 ± 0.1430	-0.2817 ± 0.2452	-	-		-	-
361247	naval_propulsion_plant	0.6970 ± 0.1294	0.9954 ± 0.0006	0.9926 ± 0.0006	RF	RF		SR	SR
361249	white_wine	0.2952 ± 0.0447	0.5097 ± 0.0387	0.4688 ± 0.0356	RF	RF		SR	SR
361250	red_wine	0.3503 ± 0.0299	0.4717 ± 0.0564	0.4308 ± 0.0514	RF	RF		SR	SR
361251	grid_stability	0.7000 ± 0.0456	0.8944 ± 0.0074	0.9294 ± 0.0071	XGB	XGB		SR	SR
361252	video_transcoding	0.7734 ± 0.0376	0.9899 ± 0.0014	0.9865 ± 0.0013	RF	RF		SR	SR
361253	wave_energy	0.7978 ± 0.0460	0.8328 ± 0.0036	0.9579 ± 0.0013	XGB	XGB		-	-
361254	sarcos	0.9272 ± 0.0031	0.9765 ± 0.0013	0.9759 ± 0.0011	-	-		SR	SR
361255	california_housing	0.6127 ± 0.0405	0.8165 ± 0.0139	0.8279 ± 0.0143	XGB	XGB		SR	SR
361256	cpu_activity	0.9713 ± 0.0070	0.9812 ± 0.0049	0.9815 ± 0.0067	-	-		SR	-
361257	diamonds	0.7899 ± 0.4962	0.9814 ± 0.0011	0.9812 ± 0.0011	-	-		-	-
361258	kin8nm	0.5241 ± 0.0327	0.7010 ± 0.0176	0.7666 ± 0.0161	XGB	XGB		SR	SR
361259	pumadyn32nh	0.4451 ± 0.2350	0.6356 ± 0.0190	0.5911 ± 0.0157	-	-		-	-
361260	miami_housing	0.7696 ± 0.0305	0.9059 ± 0.0149	0.9125 ± 0.0147	XGB	-		SR	SR
361261	cps88wages	0.2724 ± 0.0403	0.1720 ± 0.0363	0.2591 ± 0.0427	SR	-		RF	RF
361264	socmob	0.7661 ± 0.0908	0.7326 ± 0.2032	0.7815 ± 0.2623	-	-		-	-
361266	kings_county	0.7188 ± 0.0414	0.8681 ± 0.0492	0.8731 ± 0.0294	-	-		SR	SR
361267	brazilian_houses	0.6718 ± 0.3171	0.5053 ± 0.2843	0.3622 ± 0.6316	-	-		-	-
361268	fps_benchmark	0.6023 ± 0.0713	0.9958 ± 0.0005	0.9979 ± 0.0002	XGB	XGB		SR	SR
361269	health_insurance	0.3659 ± 0.0137	0.3155 ± 0.0204	0.3652 ± 0.0134	-	-		RF	RF
361272	fifa	0.7548 ± 0.0333	0.7687 ± 0.0298	0.7577 ± 0.0294	-	-		-	-
361616	Moneyball	0.9453 ± 0.0044	0.9262 ± 0.0082	0.9256 ± 0.0043	SR	SR		-	-
361617	energy_efficiency	0.9547 ± 0.0293	0.9975 ± 0.0011	0.9988 ± 0.0005	XGB	-		SR	-
361618	forest_fires	-0.2514 ± 0.3384	-4.6276 ± 8.4307	-11.4936 ± 16.6271	-	-		-	-
361619	student_performance_por	0.2208 ± 0.1170	0.2158 ± 0.1596	0.0479 ± 0.2114	-	-		-	-
361621	QSAR_fish_toxicity	0.5559 ± 0.0843	0.6011 ± 0.0669	0.5593 ± 0.0681	-	-		-	-
361622	cars	0.9136 ± 0.0308	0.9358 ± 0.0270	0.9333 ± 0.0199	-	-		-	-
361623	space_ga	0.5484 ± 0.0347	0.6487 ± 0.0618	0.6777 ± 0.0463	-	-		-	-

Table 3: Performance of selected regressors with default hyperparameters.

on OpenML-CTR23 will last a maximum of 18 days. The hyperparameter search space for the ensembles is detailed in Appendix C of the Supplementary Materials³.

4.3.1 Performance

The results of the different regressors with enhanced hyperparameters are reported in Table 4. The difference between algorithms is reduced: For $p < 0.05$, XGBoost is now the best in 8 tasks (down from 10), RF in 6 (down from 7) and SR in 2 (down from 3); as for the least-performing algorithms, at $p < 0.01$, SR is now the worst for 14 tasks (down from 16), XGBoost is the worst in 1 (same as before), while interestingly RF is never the worst (down from 2).

Results reported in Table 5 show that the $R2$ values generally improve significantly with enhanced hyperparameters, typically with larger gains for SR.

5 Discussion

5.1 LLM-assisted expert analysis of SR models

One of the stated advantages of SR over other black-box ML regression algorithms is the interpretability of the results. In order to evaluate this claim, we perform an expert analysis of the equations found during the previous experiments. As SR was run in a 10-fold cross-validation, 10 different equations were obtained for each dataset, with potentially considerable differences in structure. Our objective, for each dataset considered, is to uncover a generic equation, extracting recurring elements found in the 10 SR equations. Examining each equation to find common structural patterns by hand can be a long process, so we decide to include a Large Language Model (LLM) in the process, to perform a first screening prior to human intervention.

More in detail, the proposed expert analysis includes the following steps: (i) $\LaTeX$ files with the extracted equations are uploaded to a commercial LLM, along with a prompt asking to extract the common parts and find a generic equation with undefined constants $c_1, \dots, c_k$; (ii) one of the authors reviews the text produced by the LLM, and eventually corrects

³Supplementary materials, https://github.com/albertotonda/symbolic-regression-vs-ensembles/blob/main/tonda_lutton_2026_supplementary_materials.pdf

Symbolic Regression Can Perform on Par with Tree-based Ensembles in Regression Problems

Task ID	Dataset name	R2, mean and std			Significant best		Significant worst	
		Symbolic Regression	Random Forest	XGBoost	$p < 0.05$	$p < 0.01$	$p < 0.05$	$p < 0.01$
361234	abalone	0.5763 ± 0.0204	0.5615 ± 0.0192	0.5499 ± 0.0276	-	-	-	-
361235	airfoil_self_noise	0.8689 ± 0.0188	0.9324 ± 0.0153	0.9438 ± 0.0190	-	-	SR	SR
361236	auction_verification	0.9588 ± 0.0290	0.9928 ± 0.0015	0.9983 ± 0.0005	XGB	-	SR	-
361237	concrete_compressive_strength	0.8612 ± 0.0405	0.9046 ± 0.0287	0.9317 ± 0.0201	XGB	XGB	SR	SR
361241	physiochemical_protein	0.2813 ± 0.0142	0.6718 ± 0.0051	0.6771 ± 0.0118	-	-	SR	SR
361242	superconductivity	0.7112 ± 0.0203	0.9244 ± 0.0064	0.9234 ± 0.0081	-	-	SR	SR
361243	geographical_origin_of_music	0.0334 ± 0.1852	0.2778 ± 0.0530	0.2781 ± 0.0663	-	-	SR	-
361244	solar_flare	0.0676 ± 0.0820	0.1241 ± 0.0372	0.1184 ± 0.0324	-	-	-	-
361247	naval_propulsion_plant	0.5574 ± 0.0927	0.9957 ± 0.0006	0.9926 ± 0.0006	RF	RF	SR	SR
361249	white_wine	0.3298 ± 0.0441	0.5271 ± 0.0385	0.4748 ± 0.0400	RF	RF	SR	SR
361250	red_wine	0.3723 ± 0.0358	0.4779 ± 0.0517	0.4175 ± 0.0405	RF	-	SR	-
361251	grid_stability	0.8148 ± 0.0400	0.8970 ± 0.0068	0.9294 ± 0.0071	XGB	XGB	SR	SR
361252	video_transcoding	0.6751 ± 0.0579	0.9911 ± 0.0011	0.9921 ± 0.0019	-	-	SR	SR
361253	wave_energy	0.8374 ± 0.1784	0.8397 ± 0.0048	0.9687 ± 0.0124	-	-	-	-
361254	sarcos	0.9286 ± 0.0045	0.9796 ± 0.0013	0.9845 ± 0.0016	XGB	XGB	SR	SR
361255	california_housing	0.6292 ± 0.0637	0.8214 ± 0.0147	0.8351 ± 0.0115	XGB	-	SR	SR
361256	cpu_activity	0.9677 ± 0.0206	0.9818 ± 0.0049	0.9822 ± 0.0072	-	-	-	-
361257	diamonds	0.9565 ± 0.0199	0.9818 ± 0.0011	0.9811 ± 0.0010	RF	-	SR	-
361258	kin8nm	0.7440 ± 0.0435	0.7106 ± 0.0180	0.7666 ± 0.0161	-	-	-	-
361259	pumadyn32nh	0.6585 ± 0.0159	0.6474 ± 0.0176	0.5911 ± 0.0157	SR	SR	XGB	XGB
361260	miami_housing	0.8152 ± 0.0364	0.9103 ± 0.0137	0.9223 ± 0.0128	XGB	XGB	SR	SR
361261	cps88wages	0.2850 ± 0.0422	0.2911 ± 0.0465	0.2715 ± 0.0403	-	-	-	-
361264	socmob	0.7463 ± 0.3725	0.7347 ± 0.1798	0.8002 ± 0.1930	-	-	-	-
361266	kings_county	0.7642 ± 0.0301	0.8705 ± 0.0425	0.8847 ± 0.0353	-	-	SR	SR
361267	brazilian_houses	0.6937 ± 0.2696	0.3828 ± 0.3372	0.2067 ± 0.5606	-	-	-	-
361268	fps_benchmark	0.6586 ± 0.0613	0.9959 ± 0.0005	0.9990 ± 0.0003	XGB	XGB	SR	SR
361269	health_insurance	0.3818 ± 0.0112	0.4038 ± 0.0123	0.3738 ± 0.0195	RF	RF	-	-
361272	fifa	0.7593 ± 0.0301	0.7730 ± 0.0297	0.7665 ± 0.0248	-	-	-	-
361616	Moneyball	0.9433 ± 0.0069	0.9292 ± 0.0069	0.9329 ± 0.0062	SR	-	-	-
361617	energy_efficiency	0.9970 ± 0.0010	0.9976 ± 0.0010	0.9988 ± 0.0005	XGB	XGB	SR	SR
361618	forest_fires	-8.1797 ± 23.5817	-0.4277 ± 0.7631	-4.4231 ± 4.9086	-	-	-	-
361619	student_performance_por	0.1441 ± 0.1645	0.2861 ± 0.1282	0.2337 ± 0.2212	-	-	-	-
361621	QSAR_fish_toxicity	0.4196 ± 0.4089	0.6254 ± 0.0534	0.5933 ± 0.0569	-	-	-	-
361622	cars	0.9449 ± 0.0193	0.9406 ± 0.0182	0.9388 ± 0.0202	-	-	-	-
361623	space_ga	0.5853 ± 0.0714	0.6596 ± 0.0621	0.6777 ± 0.0463	-	-	SR	-

Table 4: Performance of selected regressors with enhanced hyperparameters.

Task ID	Dataset name	R2 PySRRRegressor (Heuristic)		R2 PySRRRegressor (Validation)		R2 RandomForestRegressor		R2 XGBRegressor	
		Default	Enhanced	Default	Enhanced	Default	Enhanced	Default	Enhanced
361234	abalone	0.5074 ± 0.0253	0.4975 ± 0.0365	0.5652 ± 0.0229	0.5763 ± 0.0204	0.5443 ± 0.0177	0.5615 ± 0.0192	0.4919 ± 0.0282	0.5499 ± 0.0276
361235	airfoil_self_noise	0.5217 ± 0.1020	0.8412 ± 0.0223	0.6364 ± 0.0620	0.8689 ± 0.0188	0.9292 ± 0.0151	0.9324 ± 0.0153	0.9466 ± 0.0152	0.9438 ± 0.0190
361236	auction_verification	0.8363 ± 0.0354	0.9576 ± 0.0167	0.8773 ± 0.0274	0.9588 ± 0.0290	0.9928 ± 0.0015	0.9928 ± 0.0015	0.9983 ± 0.0005	0.9983 ± 0.0005
361237	concrete_compressive_strength	0.7498 ± 0.0484	0.8473 ± 0.0417	0.8092 ± 0.0271	0.8612 ± 0.0405	0.8995 ± 0.0274	0.9046 ± 0.0287	0.9232 ± 0.0240	0.9317 ± 0.0201
361241	physiochemical_protein	0.1282 ± 0.0077	0.1282 ± 0.0076	0.2890 ± 0.0091	0.2813 ± 0.0142	0.6624 ± 0.0057	0.6718 ± 0.0051	0.6015 ± 0.0057	0.6771 ± 0.0118
361242	superconductivity	0.6065 ± 0.0330	0.6119 ± 0.0409	0.7113 ± 0.0203	0.7112 ± 0.0203	0.9217 ± 0.0074	0.9244 ± 0.0064	0.9176 ± 0.0066	0.9234 ± 0.0081
361243	geographical_origin_of_music	0.0878 ± 0.0521	-0.0274 ± 0.4768	0.0709 ± 0.0456	0.0334 ± 0.1852	0.2623 ± 0.0922	0.2778 ± 0.0530	0.1841 ± 0.0935	0.2781 ± 0.0663
361244	solar_flare	0.0857 ± 0.0342	0.0696 ± 0.0589	-1.5347 ± 4.8692	0.0676 ± 0.0820	-0.0980 ± 0.1430	0.1241 ± 0.0372	-0.2817 ± 0.2452	0.1184 ± 0.0324
361247	naval_propulsion_plant	0.6511 ± 0.1264	0.4132 ± 0.1367	0.6970 ± 0.1294	0.5574 ± 0.0927	0.9954 ± 0.0006	0.9957 ± 0.0006	0.9926 ± 0.0006	0.9926 ± 0.0006
361249	white_wine	0.1858 ± 0.0387	0.1650 ± 0.0360	0.2952 ± 0.0447	0.3298 ± 0.0441	0.5097 ± 0.0387	0.5271 ± 0.0385	0.4688 ± 0.0356	0.4748 ± 0.0400
361250	red_wine	0.1650 ± 0.0926	0.2407 ± 0.0903	0.3503 ± 0.0299	0.3723 ± 0.0358	0.4717 ± 0.0564	0.4779 ± 0.0517	0.4308 ± 0.0514	0.4175 ± 0.0405
361251	grid_stability	0.6438 ± 0.0718	0.7647 ± 0.0491	0.7000 ± 0.0456	0.8148 ± 0.0400	0.8944 ± 0.0074	0.8970 ± 0.0068	0.9294 ± 0.0071	0.9294 ± 0.0071
361252	video_transcoding	0.7068 ± 0.0732	0.5974 ± 0.0683	0.7734 ± 0.0376	0.6751 ± 0.0579	0.9899 ± 0.0014	0.9911 ± 0.0011	0.9865 ± 0.0013	0.9921 ± 0.0019
361253	wave_energy	0.7957 ± 0.0474	0.8090 ± 0.2395	0.7978 ± 0.0460	0.8374 ± 0.1784	0.8328 ± 0.0036	0.8397 ± 0.0048	0.9579 ± 0.0013	0.9687 ± 0.0124
361254	sarcos	0.9108 ± 0.0049	0.9100 ± 0.0099	0.9272 ± 0.0031	0.9286 ± 0.0045	0.9765 ± 0.0013	0.9796 ± 0.0013	0.9759 ± 0.0011	0.9845 ± 0.0016
361255	california_housing	0.4937 ± 0.0990	0.5461 ± 0.0935	0.6127 ± 0.0405	0.6292 ± 0.0637	0.8165 ± 0.0139	0.8214 ± 0.0147	0.8279 ± 0.0143	0.8351 ± 0.0115
361256	cpu_activity	0.9637 ± 0.0098	0.9647 ± 0.0089	0.9713 ± 0.0070	0.9677 ± 0.0206	0.9812 ± 0.0049	0.9818 ± 0.0049	0.9815 ± 0.0067	0.9822 ± 0.0072
361257	diamonds	0.9400 ± 0.0521	0.9457 ± 0.0283	0.7899 ± 0.4962	0.9565 ± 0.0199	0.9814 ± 0.0011	0.9818 ± 0.0011	0.9812 ± 0.0011	0.9811 ± 0.0010
361258	kin8nm	0.3753 ± 0.1030	0.6694 ± 0.0642	0.5241 ± 0.0327	0.7440 ± 0.0435	0.7010 ± 0.0176	0.7106 ± 0.0180	0.7666 ± 0.0161	0.7666 ± 0.0161
361259	pumadyn32nh	0.4383 ± 0.2303	0.6610 ± 0.0156	0.4451 ± 0.2350	0.6585 ± 0.0159	0.6356 ± 0.0190	0.6474 ± 0.0176	0.5911 ± 0.0157	0.5911 ± 0.0157
361260	miami_housing	0.7056 ± 0.0474	0.7571 ± 0.0530	0.7696 ± 0.0305	0.8152 ± 0.0364	0.9059 ± 0.0149	0.9103 ± 0.0137	0.9125 ± 0.0147	0.9223 ± 0.0128
361261	cps88wages	0.0962 ± 0.0178	0.1397 ± 0.0630	0.2724 ± 0.0403	0.2850 ± 0.0422	0.1720 ± 0.0363	0.2911 ± 0.0465	0.2591 ± 0.0427	0.2715 ± 0.0403
361264	socmob	0.4331 ± 0.0089	0.7125 ± 0.3993	0.7661 ± 0.0908	0.7463 ± 0.3725	0.7326 ± 0.2032	0.7347 ± 0.1798	0.7815 ± 0.2623	0.8002 ± 0.1930
361266	kings_county	0.6649 ± 0.0513	0.6908 ± 0.0359	0.7188 ± 0.0414	0.7642 ± 0.0301	0.8681 ± 0.0492	0.8705 ± 0.0425	0.8731 ± 0.0294	0.8847 ± 0.0353
361267	brazilian_houses	0.4645 ± 0.3103	0.6755 ± 0.2496	0.6718 ± 0.3171	0.6937 ± 0.2696	0.5493 ± 0.2843	0.5828 ± 0.3372	0.3622 ± 0.6316	0.2067 ± 0.5606
361268	fps_benchmark	0.4811 ± 0.0938	0.5842 ± 0.0902	0.6023 ± 0.0713	0.6586 ± 0.0613	0.9958 ± 0.0005	0.9959 ± 0.0005	0.9979 ± 0.0002	0.9990 ± 0.0003
361269	health_insurance	0.2797 ± 0.0093	0.2586 ± 0.0111	0.3659 ± 0.0137	0.3818 ± 0.0112	0.3155 ± 0.0204	0.4038 ± 0.0123	0.3652 ± 0.0134	0.3738 ± 0.0195
361272	fifa	0.6794 ± 0.0308	0.7262 ± 0.0392	0.7548 ± 0.0333	0.7593 ± 0.0301	0.7687 ± 0.0298	0.7730 ± 0.0297	0.7577 ± 0.0294	0.7665 ± 0.0248
361616	Moneyball	0.9281 ± 0.0137	0.9383 ± 0.0091	0.9453 ± 0.0044	0.9433 ± 0.0069	0.9262 ± 0.0082	0.9292 ± 0.0069	0.9256 ± 0.0043	0.9329 ± 0.0062
361617	energy_efficiency	0.9481 ± 0.0355	0.9967 ± 0.0007	0.9547 ± 0.0293	0.9970 ± 0.0010	0.9975 ± 0.0011	0.9976 ± 0.0010	0.9988 ± 0.0005	0.9988 ± 0.0005
361618	forest_fires	-11.4194 ± 31.2189	-47.3291 ± 139.8610	-0.2514 ± 0.3384	-8.1797 ± 23.5817	-4.6276 ± 8.4307	-4.6277 ± 7.631	-11.4936 ± 16.6271	-4.4231 ± 4.9086
361619	student_performance_por	0.0931 ± 0.0593	0.0327 ± 0.3776	0.2208 ± 0.1170	0.1441 ± 0.1645	0.2158 ± 0.1596	0.2861 ± 0.1282	0.0479 ± 0.2114	0.2337 ± 0.2212
361621	QSAR_fish_toxicity	0.4079 ± 0.0838	0.3501 ± 0.4786	0.5559 ± 0.0843	0.4196 ± 0.4089	0.6011 ± 0.0669	0.6254 ± 0.0534	0.5933 ± 0.0681	0.5933 ± 0.0569
361622	cars	0.8891 ± 0.0312	0.9376 ± 0.0239	0.9136 ± 0.0308	0.9449 ± 0.0193	0.9358 ± 0.0270	0.9406 ± 0.0182	0.9333 ± 0.0199	0.9388 ± 0.0202
361623	space_ga	0.4662 ± 0.0667	0.5119 ± 0.0715	0.5484 ± 0.0347	0.5853 ± 0.0714	0.6487 ± 0.0618	0.6596 ± 0.0621	0.6777 ± 0.0463	0.6777 ± 0.0463

Table 5: Regressors’ performance, with default and enhanced hyperparameters. Highlighted cells are significantly better using the Nadeau-Bengio test for $p < 0.05$.

the shape of the generic equation; (iii) the constants are optimized first using the whole dataset, to have a global equation, and then in a 10-fold cross-validation to assess overfitting. The LLM selected for the experiments is the free version of Google Gemini@ 3.1, accessed between May 19 and June 6, 2026, using the “Pro” mode with the “Extended reasoning” option. The constants have been optimized with the Covariance-Matrix-Adaptation Evolution Strategy (CMA-ES) [11], with default hyperparameters except population size set to $\mu = 100$ to improve repeatability.

We choose to analyze only the datasets for which SR obtains a mean $R^2 > 0.6$. The full results are reported in Table 6, and here in the following we highlight a few interesting cases. In task 361259 (pumadyn32nh), SR performs

significantly better than the ensembles. The equation resulting from the analysis is quite compact and elegant:

$$\hat{y} = -1.077 \cdot 10^{-3} x_{15} \cos(x_4), \quad R^2 = 0.6615 \quad (1)$$

The information on `pumadyn32nh` from the OpenML website⁴ states: “A realistic simulation of the dynamics of a Puma 560 robot arm. The task [...] is to predict the angular acceleration of one of the robot arm’s links. The task is nonlinear and has high noise. [...]”. It thus makes sense that SR, with $\cos$ and $\sin$ operators, is able to find a good correlation, while tree-based ensembles might struggle.

The equation found for task 361272 (`fifa`) uses a single feature, x_4 :

$$\hat{y} = 0.17 (x_4 - 56.201)^4 + 1823.898, \quad R^2 = 0.7568 \quad (2)$$

From the the dataset description on the OpenML website⁵, we learn that the objective of the task is to predict the wage of soccer players in the videogame FIFA, given several in-game features, and that x_4 is called `overall`, with no more information. However, searching online reveals that `overall` is the overall best relative rank of the player, providing an intuitive relationship between rank and wages, where small increases in rank can have substantial impact on the player’s salary, with respect to a baseline (the large constant term 1823.898).

On the two tasks 361249 (`white_wine`) and 361250 (`red_wine`) SR performs significantly worse than the other approaches with both default and enhanced hyperparameters: Inspecting the two datasets reveals them to be quite unusual among regression problems. Both present quantitative, real-valued features about wine (such as pH, density, content of sulfur dioxide, etc.), with the target $y \in \mathbb{N}^+$ being an assessment of quality performed by human experts. This quality assessment value ranges from 3 to 9 in `white_wine` and from 3 to 8 in `red_wine`. It is interesting to notice that the same tasks could be easily reframed as classification problems; and in fact, the ML domain defines problems of this kind as *ordinal classification* or *ordinal regression* [17], because the relative values of the integer labels have a meaning. RF and XGBoost employ similar base predictors, DTs, which ultimately predict constant values after partitioning the feature space, a characteristic which might make the tree-based ensembles more suited for these kinds of problems. Another consideration is that SR always employs MSE as its fitness function, while for ordinal regression other functions could be more suited.

Nevertheless, the highlighted differences show how different regressors might be inherently more suited for different tasks, reinforcing the conclusions of the *no free lunch* theorem for ML [26]. A more detailed description of the analysis process for each dataset is presented in Appendix E of the Supplementary Materials.

5.2 Limitations

The choice of employing SR with a wall-clock timeout (24 minutes in the experiments of Section 4) can have undesired effects: For example, when using enhanced hyperparameters, SR has a vaster search space to explore, a larger population, and more complex equations to evaluate, which naturally leads to a longer wall-clock time spent on each generation. In some cases, especially for datasets with a considerable number of samples, the timeout might prevent SR from efficiently exploring the search space, stopping the algorithm after just a few generations. Looking back at Table 4, this is what likely happens for tasks 361247 and 361252, the only two cases for which SR performs significantly *worse* with enhanced hyperparameters than with default hyperparameters. As a sanity check, we re-run the two experiments, this time employing a 5-hour timeout for SR, and we obtain a significant improvement: SR’s performance on 361247 goes from $R^2 = 0.5574 \pm 0.0927$ to $R^2 = 0.7108 \pm 0.1223$ and on 361252 goes from $R^2 = 0.6751 \pm 0.0579$ to $R^2 = 0.8491 \pm 0.0288$.

6 Conclusions

This work presented a thorough empirical evaluation of SR against state-of-the-art tree-based models, with the objectives of (RQ1) designing an experimental setup as fair as possible for all regressors, (RQ2) assessing whether SR can perform on par with the state of the art if given enough computational resources, and (RQ3) evaluating a heuristic choice against using a validation set for the final choice of model for SR. Results on a benchmark suite designed for ML regression techniques show that SR can be competitive with RF and XGBoost: Using default hyperparameters commonly found in software libraries, it often performs significantly worse, but the differences are reduced as soon as the algorithms are given access to a larger amount of computational resources. As predicted by the *no free lunch* theorem [26], SR seems to be inherently more suited for some types of problems (for example, where predictions benefit from a periodic function), while the tree-based ensembles excel in others (for example, ordinal regression tasks). Still, the results obtained so far

⁴<https://openml.org/search?type=data&status=active&id=44981>

⁵<https://openml.org/search?type=data&id=45012>

Task ID	Equation	Equation R2	Highest R2 (Algorithm)
361236	$\hat{y} = 92.431x_4 + \left(25.733x_3 - 8.269x_5 + 51.408 + \frac{36.008}{1.286-x_5}\right) \cdot \left(-72.759x_0^2 + 220.591x_0 - 1.82x_4 + 131.606 - \frac{414.259}{x_0+274.477}\right) - 6061.062$	0.8901 ± 0.0223	0.9983 ± 0.0005 (XGB)
361237	$\hat{y} = 13.342 - \frac{739.372}{x_7+17.038} + \frac{627.204x_0+477.672x_1+379.5x_2}{27.052x_3+801.264}$	0.8154 ± 0.0272	0.9317 ± 0.0201 (XGB)
361242	$\hat{y} = 4.94x_{25}x_{77} - 21.151x_{25} + 18.978x_{66} - 3.351x_{77} + \frac{1010.188x_{27}+x_{70}(19.412x_{27}+92.837x_9-2207.771)-4663.478x_9}{54.219x_{44}+(123.637x_{72}-216.296)(59.175x_{73}-42.878)+5527.946} + 11.917$	0.7217 ± 0.0143	0.9217 ± 0.0074 (XGB)
361247	$\hat{y} = 0.028x_{10} + 1.21x_{11} - 0.003x_8 + \frac{3304.762x_{11}-3446.023}{x_8(3.953x_9-9.853)+3678.592} + 1.133$	0.8020 ± 0.0035	0.9957 ± 0.0006 (RF)
361251	$\hat{y} = -0.0997 \sin(1.124 \sin(0.0750x_1x_9 + 0.06619x_{10}x_3 + 0.3753x_8 + 3.2948 + \frac{2.3662e^{-0.6165x_{11}} \log(1.0000-5.6576 \cdot 10^{-9}x_2)}{-1.9512 \cdot 10^{-8}x_0-2.0206 \cdot 10^{-7}}))$	0.6499 ± 0.0255	0.9294 ± 0.0071 (XGB)
361252	$\hat{y} = \frac{9.825 \cdot 10^{-4}x_{13}^3x_{16}-0.2125}{x_{13}-0.9828+\frac{3.8670x_{16}}{x_{14}}} + 0.9912$	0.7823 ± 0.0112	0.9921 ± 0.0019 (XGB)
361253	$\hat{y} = x_{32} + x_{33} + x_{34} + x_{35} + x_{36} + x_{37} + x_{38} + x_{39} + x_{40} + x_{41} + x_{42} + x_{43} + x_{44} + x_{45} + x_{46} + x_{47} - 105.301$	1.0000 ± 0.0000	0.9687 ± 0.0124 (XGB)
361254	$\hat{y} = 37.795x_0 - x_{10} + 1.149x_{14} - 2.975x_7x_8 + 20.163 + \frac{2.273x_{14}+1.098x_{17}}{x_3}$	0.9312 ± 0.0026	0.9759 ± 0.0011 (XGB)
361256	$\hat{y} = -0.058x_{15} - 0.03x_{17} + \frac{0.633}{1.354 \cdot 10^{-7}x_2 + \frac{1}{x_{20}} + 0.006}$	0.9708 ± 0.0068	0.9815 ± 0.0067 (XGB)
361257	$\hat{y} = (21.035x_0 - 3.686) \cdot (x_6 - 1.788) \cdot (0.605x_3(-4.107x_2 + 3.456x_6 + 4.696) + 40.006)$	0.9618 ± 0.0023	0.9812 ± 0.0011 (XGB)
361259	$\hat{y} = -1.077 \cdot 10^{-3}x_{15} \cos(x_4)$	0.6612 ± 0.0157	0.6585 ± 0.0159 (SR)
361264	$\hat{y} = 6.325x_3 + 0.936x_4 - \frac{0.24x_4}{x_1-3.701}$	0.7597 ± 0.0859	0.8002 ± 0.1930 (XGB)
361266	$\hat{y} = 31.867x_{11} + 74.53x_{12} + (20.352x_6 + 44.071x_8 - 226.401) \cdot (42.249x_{14} + x_2 + 2397.756x_5 - 2121.226) + 3828.561 \sin(21.852x_{14} + 174.102) + 1669.555$	0.6434 ± 0.0431	0.8847 ± 0.0353 (XGB)
361267	$\hat{y} = 393.222x_0 + 1399.873x_3 + 571.234x_4 + x_8 - 884.748$	0.7004 ± 0.2379	0.6718 ± 0.3171 (SR)
361268	$\hat{y} = 3.431x_1 + 2.782x_{20} + 0.873x_{29} + 0.333x_{30}x_{38} + 1.946x_{30} - 35.873x_{38}^2 + 39.939 \cos(25.355 \sin(6.623x_{36} - 3.141)) + 444.305 + \frac{371.582}{x_{36}+4.951} - \frac{20.649}{x_{36}-0.596} - \frac{380.464}{x_{38}}$	0.7066 ± 0.0099	0.9990 ± 0.0003 (XGB)
361272	$\hat{y} = 0.17(x_4 - 56.201)^4 + 1823.898$	0.7543 ± 0.0312	0.7687 ± 0.0298 (RF)
361616	$\hat{y} = 2.776x_4 + x_5(0.860x_3 + 3380.399x_6) - 150.374$	0.9454 ± 0.0048	0.9433 ± 0.0069 (SR)
361617	$\hat{y} = -0.116x_4 \left(-0.098x_2 - 26.977x_6 - \frac{0.307}{x_7-0.045} + \frac{0.082}{x_0-0.8}\right)$	0.9536 ± 0.0078	0.9988 ± 0.0005 (XGB)
361622	$\hat{y} = -\frac{89.49x_0}{-280.988x_{12}-312.119x_7+1043.105} + 4.133 \cdot 10^3(x_1 + x_{14} + x_{16}) + 1.28 \cdot 10^5(x_{10} + x_{12} + x_7) - 639.127e^{-964.96x_1+\frac{2.818 \cdot 10^3x_0}{\sin(x_2)}} - 832.179 \log(x_0) + 6.088 \cdot 10^4$	0.8909 ± 0.0296	0.9449 ± 0.0193 (SR)

Table 6: Equations designed by a human, assisted by a LLM, starting from the results obtained by SR in a 10-fold cross-validation.

seem to indicate that in a few cases SR might be able to obtain even better results with more computational effort and an even vaster search space, so future works will explore even more ambitious hyperparameter settings.

References

- [1] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery &

- Data Mining. pp. 2623–2631. Kdd '19, Association for Computing Machinery, Anchorage, AK, USA and New York, NY, USA (2019). <https://doi.org/10.1145/3292500.3330701>
- [2] Bischl, B., Casalicchio, G., Das, T., Feurer, M., Fischer, S., Gijsbers, P., Mukherjee, S., Müller, A.C., Németh, L., Oala, L., Purucker, L., Ravi, S., van Rijn, J.N., Singh, P., Vanschoren, J., van der Velde, J., Wever, M.: OpenML: Insights from 10 years and more than a thousand papers. *Patterns* **6**(7) (Jul 2025). <https://doi.org/10.1016/j.patter.2025.101317>
- [3] Breiman, L.: Random forests. *Machine learning* **45**(1), 5–32 (2001)
- [4] Breiman, L., Friedman, J., Stone, C.J., Olshen, R.A.: *Classification and Regression Trees*. CRC press (1984)
- [5] Burlacu, B., Kronberger, G., Kommenda, M.: Operon C++: An efficient genetic programming framework for symbolic regression. In: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion*. pp. 1562–1570. Gecco '20, Association for Computing Machinery, Cancún, Mexico and New York, NY, USA (2020). <https://doi.org/10.1145/3377929.3398099>
- [6] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. pp. 785–794. Kdd '16, ACM (Aug 2016). <https://doi.org/10.1145/2939672.2939785>
- [7] Cortes, C., Vapnik, V.: Support-vector networks. *Machine learning* **20**(3), 273–297 (1995)
- [8] Cranmer, M.: *Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl* (2023). <https://doi.org/10.48550/ARXIV.2305.01582>
- [9] Fischer, S.F., Feurer, M., Bischl, B.: OpenML-CTR23 – A curated tabular regression benchmarking suite. In: *AutoML Conference 2023 (Workshop)* (2023)
- [10] Grinsztajn, L., Oyallon, E., Varoquaux, G.: Why do tree-based models still outperform deep learning on typical tabular data? In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022* (2022)
- [11] Hansen, N., Ostermeier, A.: Completely derandomized self-adaptation in evolution strategies. *Evolutionary computation* **9**(2), 159–195 (2001). <https://doi.org/10.1063/1.2713540>
- [12] Hollmann, N., Müller, S., Eggensperger, K., Hutter, F.: TabPFN: A transformer that solves small tabular classification problems in a second. In: *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net (2023)
- [13] Holm, S.: A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* **6**(2), 65–70 (1979)
- [14] Jarmatz, N., Augustin, W., Scholl, S., Tonda, A., Delaplace, G.: Development of a soft sensor for fouling prediction in pipe fittings using the example of particulate deposition from suspension flow. *Food and Bioprocess Technology* **145**, 116–127 (2024). <https://doi.org/10.1016/j.fbp.2024.02.009>
- [15] Koza, J.R.: *Genetic Programming. 1: On the Programming of Computers by Means of Natural Selection*. Complex Adaptive Systems, MIT Press, Cambridge, Mass. (1992)
- [16] La Cava, W., Burlacu, B., Virgolin, M., Kommenda, M., Orzechowski, P., de França, F.O., Jin, Y., Moore, J.H.: Contemporary Symbolic Regression Methods and their Relative Performance. *Advances in neural information processing systems* **2021**(DB1), 1–16 (Dec 2021)
- [17] McCullagh, P.: Regression models for ordinal data. *Journal of the Royal Statistical Society: Series B (Methodological)* **42**(2), 109–127 (Jan 1980). <https://doi.org/10.1111/j.2517-6161.1980.tb01109.x>
- [18] Nadeau, C., Bengio, Y.: Inference for the Generalization Error. *Machine Learning* **52**(3), 239–281 (Sep 2003). <https://doi.org/10.1023/A:1024068626366>
- [19] Orzechowski, P., La Cava, W., Moore, J.H.: Where are we now?: A large benchmark study of recent symbolic regression methods. In: *Proceedings of the Genetic and Evolutionary Computation Conference*. pp. 1183–1190. ACM, Kyoto Japan (Jul 2018). <https://doi.org/10.1145/3205455.3205539>
- [20] Pearson, K.: *Mathematical Contributions to the Theory of Evolution. III. Regression, Heredity, and Panmixia*. Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character **187**, 253–318 (1896)
- [21] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011). <https://doi.org/10.5555/1953048.2078195>

- [22] Rojas-Velazquez, D.E., Tonda, A., Lopez-Rincon, A.: Biomarker modelling in omics technologies using symbolic regression. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion. pp. 895–898. GECCO '25 Companion, Association for Computing Machinery, NH Malaga Hotel, Malaga, Spain and New York, NY, USA (2025). <https://doi.org/10.1145/3712255.3726746>
- [23] Schmidt, M., Lipson, H.: Distilling Free-Form Natural Laws from Experimental Data. *Science* **324**(5923), 81–85 (Apr 2009). <https://doi.org/10.1126/science.1165893>
- [24] Sun, T.A., Kičiatovas, D., Aapalampi, I.K., Kuosmanen, T., Hiltunen, T., Mustonen, V.: Discovering data-driven microbial growth models with symbolic regression. *Methods in Ecology and Evolution* **n/a**(n/a) (2026). <https://doi.org/10.1111/2041-210x.70335>
- [25] Udrescu, S.M., Tegmark, M.: AI Feynman: A physics-inspired method for symbolic regression. *Science Advances* **6** (2019), <https://api.semanticscholar.org/CorpusID:167217655>
- [26] Wolpert, D.H.: The lack of a priori distinctions between learning algorithms. *Neural Computation* **8**(7), 1341–1390 (Oct 1996). <https://doi.org/10.1162/neco.1996.8.7.1341>